알고리즘 내부동작 시각화

셔플 알고리즘의 동작, 균등성, 효율성 비교

학번	20253067
이름	임성민
선택 주제	주제 A. 알고리즘 동작 시각화
분반	나(02)분반
비교 알고리즘	**Fisher–Yates** vs **Random Pick & Remove** (+ Riffle Shuffle, Random-Key Sort)
웹앱	https://shuffle-lab-pi.vercel.app
소스 코드	github.com/Ricotta-Cheese/ShuffleLab

1. 주제와 과제 설명

1.1 주제를 고른 이유

학교에서 여러 종류를 배우는 정렬 알고리즘들과는 반대로, 배열을 무작위로 섞는 셔플은 사용할 일이 많지 않았습니다. 그래서 셔플 알고리즘에도 여러 방법이 있는지, 성능 차이는 어떠한지 직접 확인하고 싶었습니다. 이번 과제에서는 셔플 알고리즘을 구현하고 동작 과정, 실행 시간, 결과 분포를 비교했습니다.

배열이 원본과 전혀 다르게 섞였다고 해서 모든 순열이 같은 확률로 나오는 것도 아니고, 반대로 완벽히 균등한 셔플을 돌려도 우연히 원본과 비슷한 순서가 나올 때가 있습니다. 따라서 동작 과정, 균등성, 효율성을 별도로 분석했습니다.

1.2 문제 정의

셔플은 서로 다른 원소 n개로 구성된 배열의 순서를 무작위로 바꾸되, 원소가 중복되거나 빠지지 않도록 하는 문제입니다.

균등성은 n개의 원소를 섞을 때 가능한 결과는 $n!$이므로, 이 결과들이 각각 $1/n!$의 확률로 나오는지를 기준으로 삼았습니다.

관계성은 Kendall's τ를 이용해 섞인 배열이 기존 원본과 얼마나 비슷한 순서 관계를 유지하는지 측정했습니다. 다만 확률적으로 셔플의 전후 결과가 일치할 수도 있으므로, 셔플을 여러 번 반복 후 평균값을 사용하도록 하였습니다.

효율성은 실행 시간과 추가 메모리 사용량으로 비교했습니다.

1.3 과제 구성

Shuffle Lab에는 네 가지 셔플 알고리즘을 구현했습니다. 핵심 비교 대상으로는 **Fisher–Yates**와 **Random Pick & Remove**를 골랐습니다. 두 방법 모두 이론적으로 균등한 순열을 만들지만, 원소를 옮기는 방식이 달라 효율성을 비교하기 좋기 때문입니다. Riffle Shuffle과 Random-Key Sort는 셔플 방법과 결과 분포를 더 넓게 살펴보기 위한 비교 대상으로 추가했습니다.

화면	내용
A 설명	주제, 연구 가설, 참고 문헌
B 알고리즘	원리, 의사코드, 복잡도 비교표와 이유
C 동작 시각화	원소 2~10개로 한 단계씩 실행, 단계마다 "무엇을, 왜" 설명
D 관계성	Kendall's τ로 결과가 원본과 얼마나 닮았는지 측정

화면	내용
E 균등성	반복 실험으로 순열 분포와 균등분포의 차이 (TVD) 측정
F 효율성	배열 크기를 2배씩 늘리며 실행 시간 측정
G 전체 비교	D, E, F 결과를 한 표로 모음

구현은 실제 셔플을 수행하는 `shuffle`과 단계별 상태를 기록하는 `trace`로 나눴습니다. 시각화에는 기록이 필요하지만, 실행 시간까지 기록하면서 재면 알고리즘 자체의 성능을 비교하기 어렵기 때문입니다. 성능 실험에는 `shuffle`만 사용했습니다.

1.4 연구 가설

1. 시간 복잡도가 같아도 셔플 결과의 균등성은 다를까? (Fisher–Yates vs Riffle Shuffle)
2. 둘 다 균등하게 섞는 알고리즘이라면 속도와 메모리 사용량도 비슷할까? (Fisher–Yates vs Random Pick & Remove)
3. 원본과 순서가 크게 달라진 결과를 더 좋은 셔플이라고 볼 수 있을까?
4. 균등한 셔플에서도 원본과 비슷한 결과가 얼마나 자주 나타날까?

2. 알고리즘 설명

2.1 Fisher–Yates (Durstenfeld 방식)

Fisher–Yates 방식은 배열의 뒤쪽부터 원소의 자리를 하나씩 확정합니다. 현재 위치가 i라면 아직 확정되지 않은 0~i 범위에서 위치 j를 무작위로 고르고, $A[i]$와 $A[j]$를 교환합니다.

- 교환이 끝난 i 번 칸은 다음 선택 범위에서 제외됩니다.
- 따라서 앞부분의 선택 범위가 매번 한 칸씩 줄어듭니다.
- 한 단계에서 하는 일은 난수 선택 한 번과 교환 한 번뿐입니다.

2.2 Random Pick & Remove

Random Pick & Remove는 남아 있는 원소에서 하나를 뽑아 새로운 배열 끝에 넣는 방식입니다. 카드를 한 장씩 뽑아서 다른 줄에 놓는 과정과 비슷합니다.

입력 배열을 복사한 뒤, 남은 원소가 len개일 때 0~len−1 중 한 위치를 균등하게 고릅니다. 선택한 값을 결과 배열에 넣고, 원래 자리 뒤에 있던 원소들을 한 칸씩 앞으로 당겨 제거합니다. 같은 원소가 다시 선택되지는 않지만, 중간 원소를 지울 때마다 여러 값을 옮겨야 한다는 점이 Fisher–Yates와 다릅니다.

2.3 확장 비교

- **Riffle Shuffle**: 카드를 두 더미로 나눈 뒤, 각 더미 안의 상대적인 순서를 유지하면서 합치는 방법입니다. Gilbert–Shannon–Reeds 모델을 사용해 이 과정을 7 번 반복하도록 구현했습니다. 한 번 섞을 때 동전 n 번의 결과로 자르는 위치와 합치는 순서를 정합니다. 반복 횟수가 고정되어 시간 복잡도는 O(n)이지만, 모든 순열이 정확히 같은 확률로 나오지는 않습니다. 특히 배열 크기가 달라지면 균등분포와의 차이도 달라집니다.
- **Random-Key Sort**: 각 원소에 [0, 1) 범위의 무작위 키를 붙이고 키를 기준으로 정렬합니다. 키가 서로 독립적이고 모두 다르다면 각 순열이 같은 확률로 나옵니다. 다만 난수 키는 낮은 확률이지만 중복이 가능하고, 안정 정렬에서 같은 키의 원래 순서가 유지되면 편향이 생길 수 있습니다. 정렬 과정 때문에 시간 복잡도는 O(n log n)입니다.

2.4 난수

난수 생성 방식에 따라 실험 결과가 달라지는 것을 구분하려고, 모든 알고리즘이 같은 난수 인터페이스를 사용하도록 했습니다. 기본값은 시드를 고정할 수 있는 SFC32입니다. 입력 배열과 시드가 같으면 같은 과정을 다시 확인할 수 있습니다.

- `rng.int(k)`: 0 부터 k−1 까지의 정수를 선택합니다. 32 비트 난수를 단순히 k 로 나눈 나머지로 처리하면 모듈러 편향에 의해 특정 값이 조금 더 자주 나올 수 있습니다. 이를 피하려고 아래의 경계 이상인 값은 버리고 다시 뽑는 거부 샘플링을 사용했습니다.

$$limit = 2^{32} - (2^{32} \bmod k), \quad u < limit \Rightarrow u \bmod k$$

- `rng.float()`: 32 비트 정수 두 개를 이용해 53 비트 정밀도의 실수를 만들며, Random-Key Sort 의 키에 사용합니다.

3. 핵심 의사코드

Fisher–Yates 의사코드

```
for i ← n−1 down to 1
    j ← random integer, 0 ≤ j ≤ i      // 아직 확정되지 않은 칸 중 하나
    swap A[i], A[j]                     // i번 칸 확정
```

Random Pick & Remove 의사코드

```
remaining ← copy of A,  result ← empty
while remaining is not empty
    k ← random integer, 0 ≤ k < length(remaining)
    append remaining[k] to result
    remove remaining[k]                 // 뒤쪽 원소를 한 칸씩 당김
```

4. 복잡도 비교

알고리즘	시간	추가 공간	입력 배열 직접 갱신	균등성
Fisher–Yates	O(n)	O(1)	예	균등
Random Pick & Remove	$O(n^2)$	O(n)	아니요	균등
Riffle Shuffle (7회)	O(n)	O(n)	예	균등분포에 근접(작은 n)
Random-Key Sort	O(n log n)	O(n)	아니요	키 중복이 없으면 균등

4.1 복잡도와 균등성의 근거

시간

- Fisher–Yates 는 n−1 번 반복하며, 각 반복에서 난수를 한 번 뽑고 두 원소를 교환합니다. 반복 한 번의 비용이 일정하므로 O(n)입니다.
- Pick & Remove 는 n 번 원소를 뽑지만, 뽑은 자리를 비우기 위해 뒤쪽 값을 옮깁니다. 남은 원소가 len 개일 때 한 번에 최대 len−1 개를 옮기며, 선택 위치가 균등하면 평균 이동 횟수는 (len−1)/2 개입니다. 전체 평균 이동 횟수는 다음과 같습니다.

$$\sum_{len=1}^{n} \frac{len-1}{2} = \frac{n(n-1)}{4} = O(n^2)$$

공간

- Fisher–Yates: 입력 배열 안에서 교환하고 임시 변수 하나만 사용하므로 추가 공간은 O(1)입니다.

- Pick & Remove: 입력 복사본인 remaining 과 별도의 결과 배열을 만들어 반환하므로 추가 공간은 O(n)이며 원본 배열은 갱신하지 않습니다. 반면 Riffle Shuffle 은 합치는 과정에서 O(n) 보조 배열을 사용하되 최종 결과를 입력 배열에 덮어쓰도록 구현했습니다.

균등성

- 각 단계에서 독립적이고 균등하게 원소를 고른다고 가정하면, Fisher–Yates 와 Pick & Remove 는 모두 가능한 선택 경로가 n!개입니다. 경로 하나가 결과 순열 하나에 대응하므로 각 순열의 이론적 확률은 1/n!입니다. 물론 이는 이상적인 난수를 가정했을 때의 이론적 성질이며, 실제 구현에서는 의사난수 생성기(PRNG)의 주기와 상태 크기에 영향을 받습니다.
- Riffle Shuffle 은 매회 동전 n 번을 던지고 이를 7 번 반복하므로 2^(7n)개의 선택 경로가 있습니다. n ≥ 3 일 때 n!에는 인수 3 이 있지만 2^(7n)에는 없어서, 모든 순열에 경로를 똑같이 배분할 수 없습니다. 즉, 이 구현의 결과 분포는 정확히 균등하지 않습니다.

5. 동작 시각화

두 알고리즘에 입력 배열 [1, 2, 3, 4, 5, 6]과 SFC32 시드 1을 사용했습니다. 처음 다섯 번의 선택값은 2, 3, 1, 0, 0으로 같지만 처리 방식이 달라 결과가 달라집니다.

5.1 Fisher–Yates 단계별 동작



그림 1. Fisher–Yates 단계 다이어그램 (실제 실행 기록으로 생성, 시드 1)

단계	수행한 동작	배열 (굵게 = 확정)	동작의 이유
0	시작	1 2 3 4 5 6	맨 뒤 칸부터 하나씩 확정합니다. 처음에는 0~5 전체가 아직 정해지지 않은 범위입니다.
1	i = 5, 0~5에서 j = 2를 골라 A[2]=3 ↔ A[5]=6 교환	1 2 6 4 5 **3**	아직 정해지지 않은 6칸 중 하나를 1/6 확률로 골라 5번 칸에 올 원소를 정합니다.
2	i = 4, 0~4에서 j = 3을 골라 A[3]=4 ↔ A[4]=5 교환	1 2 6 5 **4 3**	선택 범위를 0~4로 줄여 이미 확정된 5번 칸이 다시 바뀌지 않게 합니다.
3	i = 3, 0~3에서 j = 1을 골라 A[1]=2 ↔ A[3]=5 교환	1 5 6 **2 4 3**	남은 4칸 중 하나를 1/4 확률로 골라 3번 칸을 정합니다.
4	i = 2, 0~2에서 j = 0을 골라 A[0]=1 ↔ A[2]=6 교환	6 5 **1 2 4 3**	남은 3칸 중 하나를 1/3 확률로 골라 2번 칸을 정합니다.
5	i = 1, 0~1에서 j = 0을 골라 A[0]=6 ↔ A[1]=5 교환	**5 6 1 2 4 3**	마지막 두 칸을 정하면 0번 칸도 자동으로 확정됩니다. 반복 횟수는 n−1 = 5번입니다.

최종 결과는 **[5, 6, 1, 2, 4, 3]**입니다. 난수 호출과 교환은 각각 5번이며, 셔플 자체에는 추가 배열이 필요하지 않습니다.

5.2 Random Pick & Remove 단계별 동작

단계	수행한 동작	남은 배열 → 결과	동작의 이유 (당긴 원소 수)
0	시작	1 2 3 4 5 6 → (빈 배열)	원본을 복사해 두고 빈 결과 배열을 만듭니다.
1	남은 6개 중 2번 칸(값 3)을 뽑음	1 2 4 5 6 → 3	각 원소를 1/6 확률로 고릅니다. 3을 지운 뒤 뒤쪽 원소 3개를 당깁니다. (3)
2	남은 5개 중 3번 칸(값 5)을 뽑음	1 2 4 6 → 3 5	남은 원소 중에서만 고르므로 같은 원소가 두 번 나오지 않습니다. (1)
3	남은 4개 중 1번 칸(값 2)을 뽑음	1 4 6 → 3 5 2	앞쪽 원소를 뽑을수록 당겨야 할 원소가 많아집니다. (2)
4	남은 3개 중 0번 칸(값 1)을 뽑음	4 6 → 3 5 2 1	맨 앞을 지우면 뒤에 남은 원소를 모두 당깁니다. (2)
5	남은 2개 중 0번 칸(값 4)을 뽑음	6 → 3 5 2 1 4	첫 번째 원소를 제거하므로 뒤에 있던 6을 한 칸 당깁니다. (1)

6	남은 1개(값 6)를 뽑음	(빈 배열) → 3 5 2 1 4 6	마지막 원소이므로 당길 원소가 없습니다. (0)



그림 2. Random Pick & Remove 단계 다이어그램 (실제 실행 기록으로 생성, 시드 1)

최종 결과는 **[3, 5, 2, 1, 4, 6]**입니다. 난수 호출은 6번, 원소 이동은 9번입니다. 입력 복사본과 결과 배열(`remaining`, `result`)을 따로 사용합니다.

5.3 두 과정의 차이

두 알고리즘 모두 남은 후보 중 하나를 고릅니다. Fisher–Yates는 현재 칸과 교환하지만, Pick & Remove는 결과 배열에 추가한 뒤 빈자리를 메우기 위해 다른 원소들을 이동시킵니다.

예제에서 Fisher–Yates는 5번 교환하고 Pick & Remove는 원소를 9번 당겼습니다. 두 연산의 비용이 같지는 않지만, 전체 교환 횟수는 n−1회, 평균 이동 횟수는 n(n−1)/4회라는 차이가 있습니다.

6. 실험 결과

6.1 효율성 (H2)

배열 크기를 256부터 두 배씩 늘리면서 셔플 1회에 걸리는 시간을 비교했습니다.

배열 생성과 사전 복사는 측정에서 제외하되, 알고리즘 내부의 난수 생성과 메모리 할당은 포함했습니다. 짧은 실행은 여러 번 묶어 재고, 전체 시간을 실행 횟수로 나누었습니다.

크기 n	Fisher–Yates	Pick & Remove	비율
1,024	39.1 µs	387.5 µs	9.9배
4,096	135.9 µs	5.10 ms	37.5배
16,384	581.3 µs	78.1 ms	134배
32,768	1.10 ms	316.2 ms	287배

실험에는 SFC32(시드 1)를 사용했고, 각 크기에서 9회 측정한 값의 중앙값을 기록했습니다. 한 크기의 중앙값이 200ms를 넘으면 그보다 큰 입력은 측정하지 않도록 했습니다. 그래서 Pick & Remove는 n = 32,768에서 316.2ms가 나온 뒤 n = 65,536부터 측정을 중단했습니다.

마지막 측정 구간(n = 16,384 → 32,768)에서 Fisher–Yates는 약 1.89배, Pick & Remove는 약 4.05배 느려졌습니다. 선형 알고리즘의 2배 증가, 이차 알고리즘의 4배 증가와 가까운 값입니다. 두 알고리즘의 실행 시간 차이도 n = 1,024에서 9.9배였지만 n = 32,768에서는 287배로 커졌습니다.

직접 측정한 범위에서는 이론적으로 예상한 시간 복잡도 차이가 실제 실행 시간에도 나타났습니다. 특히 Pick & Remove는 배열이 커질수록 삭제 후 원소를 당기는 비용이 커졌습니다.

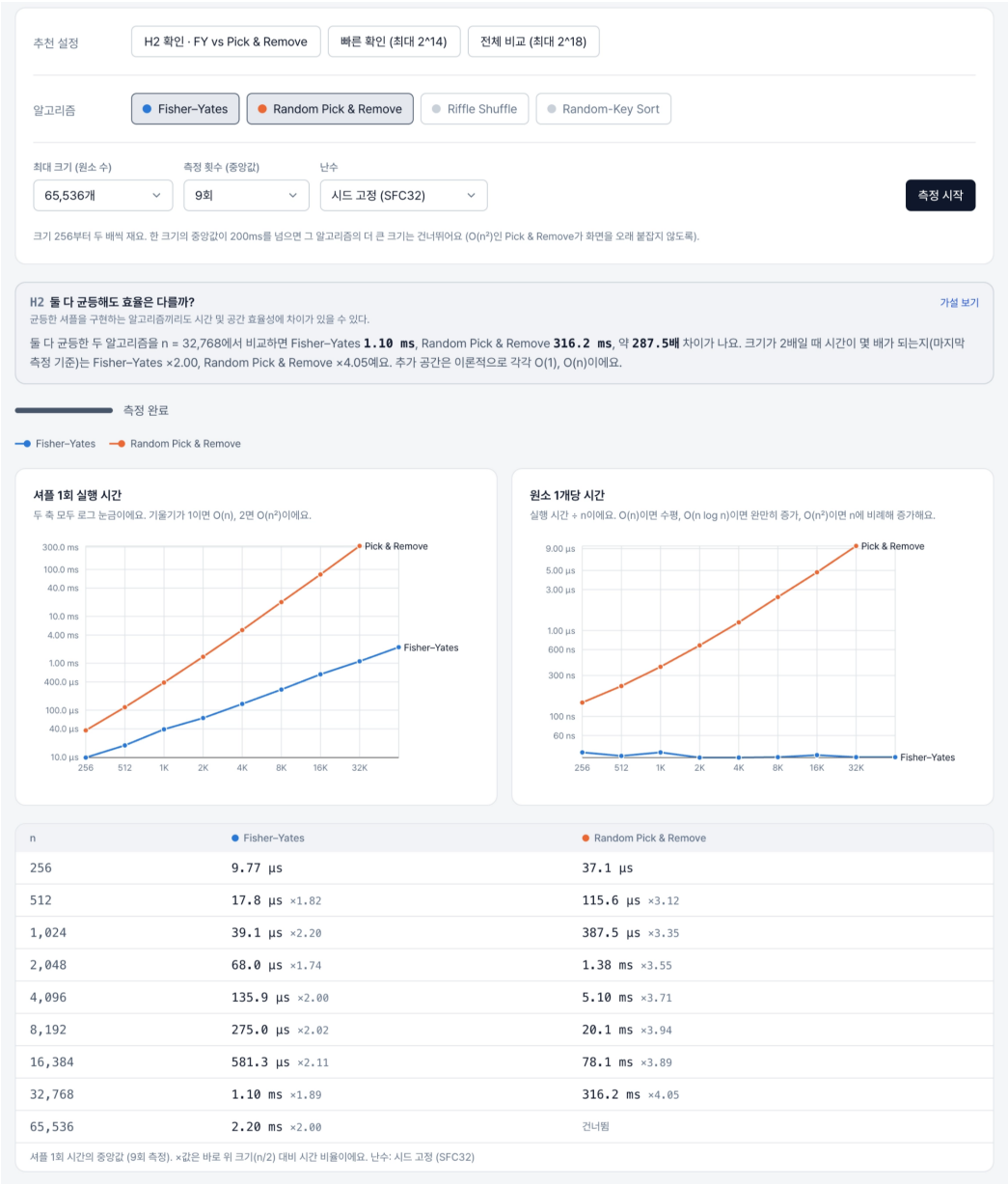


그림 3. 웹앱 F 화면, H2 확인 실험(Fisher–Yates vs Pick & Remove). 왼쪽은 셔플 1회 시간(로그-로그), 오른쪽은 원소 1개당 시간을 보여줍니다.

6.2 균등성 (H1)

셔플 결과가 고르게 나오는지는 같은 배열을 여러 번 섞어 확인해보았습니다.

순열별 발생 횟수를 전체 실행 횟수로 나누어 관측 확률을 구하고, 균등분포 1/n!과의 차이를 총변동거리(TVD)로 계산했습니다. 아래 식의 P에 관측 분포를 넣으면 관측 TVD, 알고리즘의 이론 분포를 넣으면 이론 TVD가 됩니다.

$$\mathrm{TVD} = \tfrac{1}{2} \Sigma \left| P(\pi) - 1/n! \right| \quad \text{(전체 순열 } \pi \text{에 대해 합산)}$$

알고리즘	n	반복 횟수	관측 TVD	이론 TVD
Fisher–Yates	4	1,000,000	0.0013	0
Random Pick & Remove	4	1,000,000	0.0016	0

알고리즘	n	반복 횟수	관측 TVD	이론 TVD
Riffle Shuffle	4	1,000,000	0.0082	0.0091
Random-Key Sort	4	1,000,000	0.0022	0 (키가 모두 다를 때)

입력 원소는 4개, 난수는 SFC32(시드 1)로 고정하고 알고리즘별로 100만 번씩 실행했습니다. 가능한 순열은 24가지이므로 균등할 때 각 순열의 확률은 1/24(약 4.17%)입니다. Fisher–Yates와 Pick & Remove는 정수 선택 경로를 열거해 이론 분포를 구했고, Riffle Shuffle은 Bayer–Diaconis의 상승 수열 공식을 사용했습니다. Random-Key Sort의 이론값은 키가 독립적이고 서로 다르다는 가정 아래 계산한 것으로, 실제 키 중복에 따른 편향은 반영하지 않았습니다.

Fisher–Yates와 7회 Riffle Shuffle은 모두 O(n)이지만, 원소 4개에서 이론 TVD는 각각 0과 약 0.0091입니다. 실제 실험값도 각각 0.0013과 0.0082였습니다. 따라서 같은 선형 시간 알고리즘이라도 순열이 나오는 확률은 다를 수 있다는 H1을 뒷받침합니다.

Fisher–Yates와 Pick & Remove는 이론적으로 균등해도 관측 TVD가 0은 아니었습니다. 100만 번을 반복해도 24개 순열이 정확히 같은 횟수로 나오지는 않으며, 완전 균등분포에서도 표본 오차로 인해 약 0.0019 수준의 기대 TVD가 발생하기 때문입니다. 즉 유한 번의 실험에서는 진짜 균등한 알고리즘도 표본 오차 때문에 TVD가 0보다 크게 나오므로, 관측된 수치 자체보다 그 값이 표본 오차 범위 안에 있는지를 함께 봐야 합니다.

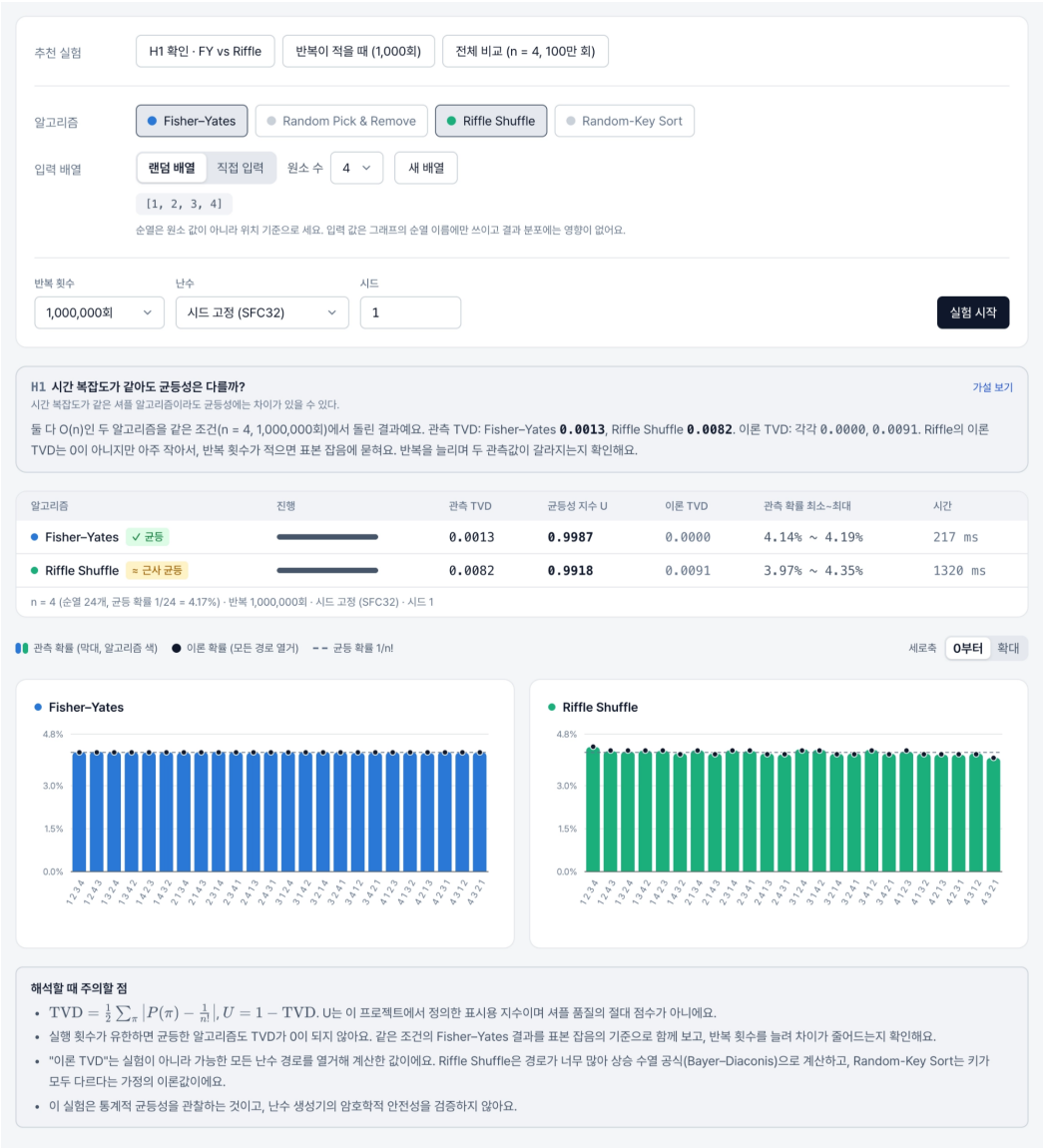


그림 4. 웹앱 E 화면, H1 확인 실험. 막대는 순열별 관측 확률, 점은 이론 확률, 점선은 균등 확률 1/24을 나타냅니다.

6.3 관계성 (H3, H4)

배열 하나를 섞었을 때 원본과의 순서 관계는 Kendall's τ로 계산했습니다. 두 원소의 앞뒤 순서가 원본과 같으면 일치(C), 반대면 불일치(D)로 셉니다.

$$\tau = \frac{C - D}{C + D}$$

결과 (원소 8개, Fisher–Yates, 시드 1)

- 원소 8 개를 시드 1 로 한 번 섞었을 때는 C = 16, D = 12 였고, τ = (16 − 12) / 28 = 0.143 이었습니다.
- 반복 실험에서는 단일 실행과 난수 수열을 구분하기 위해 시드를 1,000,004(= 1 + 1,000,003)로 바꾸어 2,000 번 섞었습니다. 평균 τ 는 약 −0.010 이었지만, |τ| ≥ 0.5 인 결과도 **10.7%** 나왔습니다. 평균값이 0 에 가깝다고 해서 매번 원본과 무관한 순서만 나오는 것은 아닙니다(H4).

- 반대로 원소 8 개를 '항상 3 칸 회전'시키면 τ = −0.071 로 0 에 가깝습니다. 하지만 이 규칙은 40,320 가지 가능한 순열 중 한 가지만 만듭니다. 이론 분포로 계산한 균등성 지수 U 는 2.48×10^{-5}에 불과합니다. 따라서 원본과의 순서 관계가 약하다는 이유만으로 균등한 셔플이라고 할 수는 없습니다(H3).

웹앱에서는 원본과의 순서 관계를 R = |τ|, 균등분포와의 가까움을 U = 1 − TVD로 표시했습니다. U는 비교를 쉽게 하려고 이번 프로젝트에서 정의한 지수입니다. R은 단일 결과에서 계산하고 U는 결과의 확률분포에서 계산하므로, 두 수치를 같은 의미의 '무작위성 점수'로 해석해서는 안 됩니다.

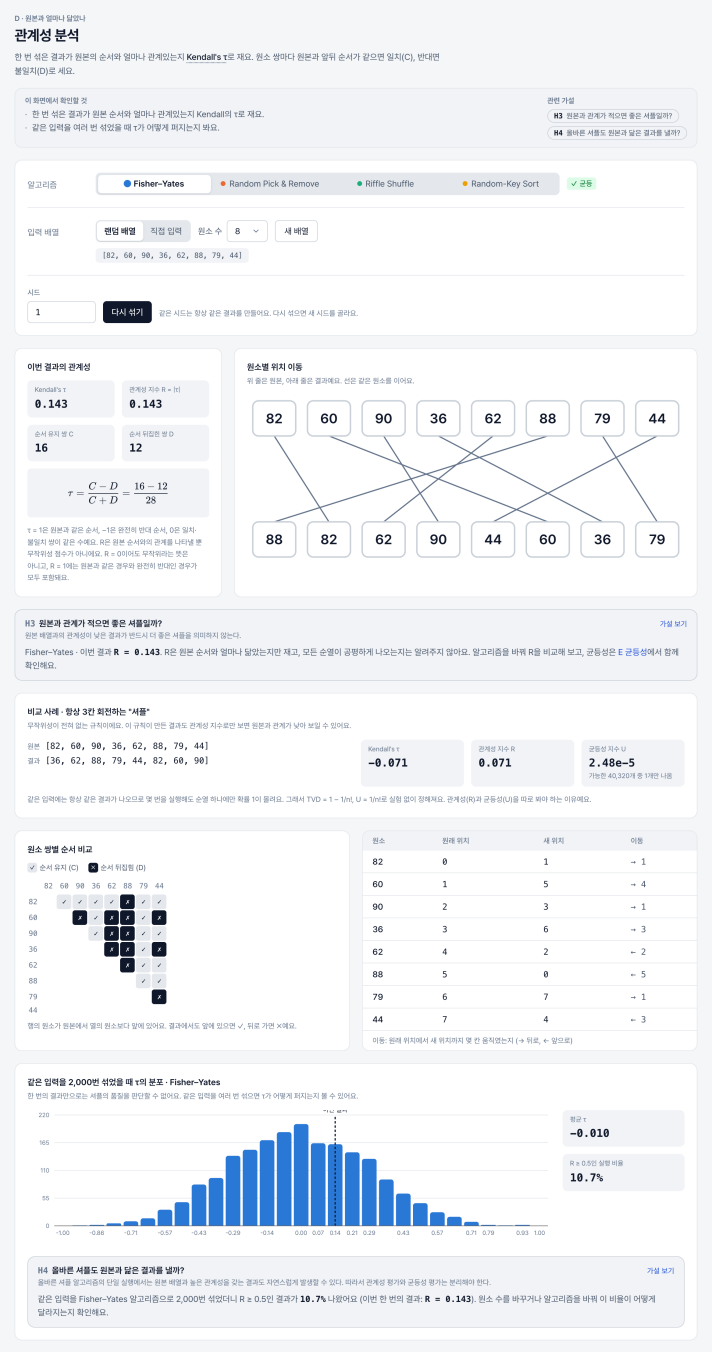


그림 5. 웹앱 D 화면. 원소별 위치 이동, 원소 쌍별 순서 비교, 3칸 회전 비교 사례, 2,000번 섞었을 때의 τ 분포

6.4 전체 비교

G 화면에서는 관계성, 균등성, 효율성 결과를 한 번에 비교할 수 있도록 모았습니다. 기존 종합 비교 기록은 관계성 n = 8, 2,000회, 균등성 n = 4, 100만 회, 효율성 n = 16,384까지 크기별 5회 측정이라는 조건으로 만들었습니다. 기준 시드는 1이고 관계성 반복 실험에는 앞서 설명한 시드 오프셋을 적용했습니다.

이 종합 기록에서는 n = 16,384에서 Pick & Remove가 Fisher–Yates보다 142.9배 느렸습니다. 6.1절의 134배와 다른 이유는 별도 측정이며 반복 횟수도 다르기 때문입니다. 평균 R은 네 알고리즘 모두 약 0.231~0.234로, n = 8 완전 무작위 순열의 이론적 기댓값(약 0.230)과 비슷했습니다. 같은 R 값대에서도 속도와 균등성이 다를 수 있다는 점을 확인했습니다.

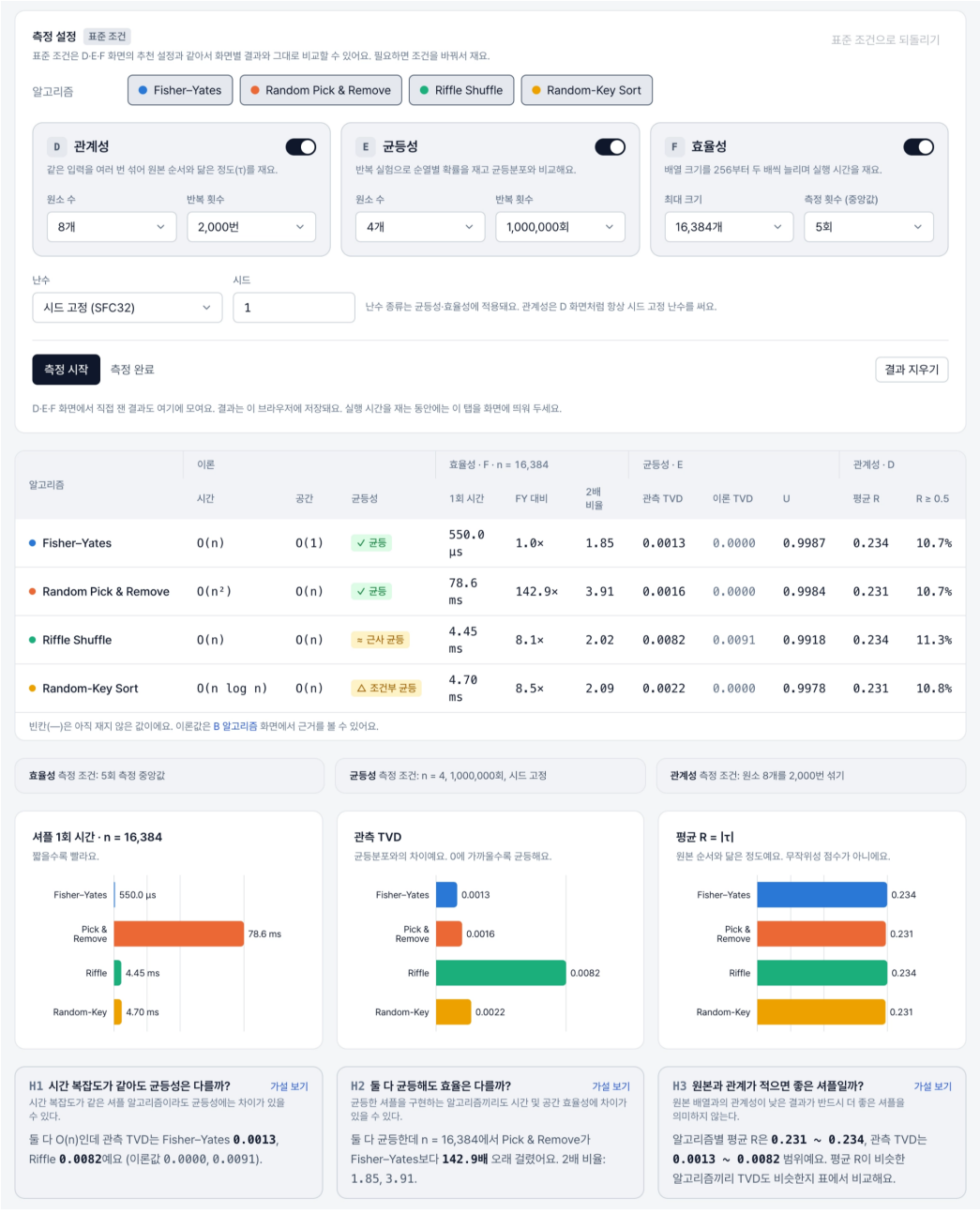


그림 6. 웹앱 G 화면, 네 알고리즘의 관계성, 균등성, 효율성 종합 비교

7. 결론

실험 결과, 구현한 네 가지 방법 중 균등성과 성능을 모두 만족하는 것은 Fisher–Yates였습니다. 특히 Fisher–Yates와 Random Pick & Remove를 비교해 보면, 매 단계 균등하게 원소를 뽑는다는 원리는 같아도 선택한 원소를 처리하는 연산 방식에서 큰 차이가 났습니다.

가장 큰 차이는 무작위로 원소를 고른 다음에 어떤 연산을 하느냐였습니다. Fisher–Yates와 Random Pick & Remove는 각 단계에서 균등하게 선택한다는 가정 아래 모두 n!가지 순열을 같은 확률로 만들 수 있습니다. 하지만 Fisher–Yates는 현재 위치와 선택한 위치를 교환하는 반면, Pick & Remove는 선택한 값을 꺼낸 뒤 남은 원소를 계속 당깁니다. 그래서 현재 구현의 시간 복잡도는 각각 $O(n)$, $O(n^2)$이고 추가 공간도 $O(1)$, $O(n)$으로 차이가 납니다.

실행 시간 측정에서도 이 차이가 나타났습니다. n을 16,384에서 32,768로 늘렸을 때 실행 시간은 Fisher–Yates가 약 1.89배, Pick & Remove가 약 4.05배로 증가했습니다. $O(n)$과 $O(n^2)$이라는 이론적 시간 복잡도 차이가 실제 실행 시간에서도 그대로 확인되었습니다(H2).

시간 복잡도가 같아도 셔플 결과의 분포는 달라질 수 있다는 점도 확인했습니다. Fisher–Yates와 7회 Riffle Shuffle은 모두 $O(n)$이지만, 원소 4개에서 이론 TVD가 각각 0과 약 0.0091이었습니다(H1). 또한 원본을 항상 3칸 회전시키는 경우에는 Kendall's τ가 0에 가까워도 결과가 한 가지로 고정됩니다(H3). 반대로 Fisher–Yates의 반복 실험에서는 $|\tau|$가 큰 결과도 나왔습니다(H4). 따라서 '원본과 얼마나 달라졌는가'는 '모든 순열이 얼마나 고르게 나오는가'와 별도로 봐야 한다는 것을 알았습니다.

8. AI 활용

웹앱 백엔드, 프론트엔드 구현과 시각화에 Opus 5.5를, 실험 진행에 필요한 지식 질문 및 실험 결과 정리에 GPT-6.1 Sol을, 보고서 윤문에 GPT-6 Astra와 Gemini 4 Argon을 사용했습니다.

9. 참고 문헌

1. Bayer, D., & Diaconis, P. (1992). Trailing the dovetail shuffle to its lair. *The Annals of Applied Probability, 2*(2), 294–313. https://doi.org/10.1214/aoap/1177005705
2. Doty-Humphrey, C. (n.d.). *PractRand: Practically random* [Computer software]. https://pracrand.sourceforge.net/
3. Durstenfeld, R. (1964). Algorithm 235: Random permutation. *Communications of the ACM, 7*(7), 420. https://doi.org/10.1145/364520.364540
4. Fisher, R. A., & Yates, F. (1938). *Statistical tables for biological, agricultural and medical research*. Oliver & Boyd.
5. Kendall, M. G. (1938). A new measure of rank correlation. *Biometrika, 30*(1–2), 81–93. https://doi.org/10.1093/biomet/30.1-2.81

6. Knuth, D. E. (1997). *The art of computer programming: Vol. 2. Seminumerical algorithms* (3rd ed.). Addison-Wesley.
7. Lemire, D. (2019). Fast random integer generation in an interval. *ACM Transactions on Modeling and Computer Simulation, 29*(1), Article 3. https://doi.org/10.1145/3230636
8. Levin, D. A., & Peres, Y. (2017). *Markov chains and mixing times* (2nd ed.). American Mathematical Society.
9. MDN Web Docs. (n.d.). *Crypto: getRandomValues() method*. https://developer.mozilla.org/en-US/docs/Web/API/Crypto/getRandomValues
10. MDN Web Docs. (n.d.). *Math.random()*. https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/random
11. Steele, G. L., Jr., Lea, D., & Flood, C. H. (2014). Fast splittable pseudorandom number generators. *Proceedings of the 2014 ACM International Conference on Object Oriented Programming Systems Languages & Applications*, 453–472. https://doi.org/10.1145/2660193.2660195